

EXPERIMENT 8

(A)

A company maintains customer information, product information, and sales order information in three separate tables: customer_master, product_catalog, and sales_orders. The company wants to provide an order report to an external client. The report must contain the **Order ID, Order Date, Customer Name, Product Name, and Final Amount**. For every order, first calculate the **Gross Amount** using: **Gross Amount = Unit Price × Quantity**. Then calculate the **Discount Amount** using: **Discount Amount = Gross Amount × Discount Percentage / 100**

Finally, calculate the **Final Amount** using: **Final Amount = Gross Amount – Discount Amount**. Create a **complex view** using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage.

CODE:

```
CREATE OR REPLACE VIEW vW_PRODUCT_DISCOUNT_INFO
```

```
AS
```

```
SELECT
```

```
    O.order_id,
```

```
    O.order_date,
```

```
    C.full_name,
```

```
    P.product_name,
```

```
    (P.unit_price * O.quantity)
```

```
    -
```

```
    (
```

```
        (P.unit_price * O.quantity)
```

```
        * O.discount_percent / 100
```

```
    ) AS final_amount
```

```
FROM customer_master C
```

JOIN sales_orders O

ON C.customer_id = O.customer_id

JOIN product_catalog P

ON P.product_id = O.product_id;

	order_id integer	order_date date	full_name character varying (50)	product_name character varying (50)	final_amount numeric
1	1	2025-09-01	Amit Sharma	Smartphone X100	47500.0000000000000000
2	2	2025-09-02	Priya Verma	Laptop Pro 15	58500.0000000000000000
3	3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.0000000000000000
4	4	2025-09-04	Neha Singh	Smartwatch Fit	27600.0000000000000000
5	5	2025-09-05	Arjun Mehta	Gaming Console	39600.0000000000000000
6	6	2025-09-06	Priya Verma	Smartphone X100	23750.0000000000000000
7	7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.0000000000000000

(B) An e-commerce company wants to regularly analyze the performance of its **product categories**.

The company stores product, category, customer, and order information in separate tables. To generate the category performance report, the database needs to perform multiple **JOINS, GROUP BY, aggregate functions, DISTINCT counting, calculated values, and CASE statements**.

Since the same report is generated frequently, executing the complete analytical query every time can increase database workload.

Create a PostgreSQL **Materialized View** named: mv_category_sales

CODE:

CREATE MATERIALIZED VIEW mv_category_sales_01

AS

SELECT

c.category_id,

c.category_name,

SUM(od.quantity) AS total_products_sold,

```
COUNT(DISTINCT od.order_id) AS total_orders,

COUNT(DISTINCT od.customer_id) AS unique_customers,

SUM(od.quantity * od.unit_price) AS total_revenue,

ROUND(
    SUM(od.quantity * od.unit_price)
    / COUNT(DISTINCT od.order_id),
    2
) AS avg_revenue_per_order,

CASE
    WHEN SUM(od.quantity * od.unit_price) >= 10000
        THEN 'Excellent'

    WHEN SUM(od.quantity * od.unit_price) >= 6000
        THEN 'Good'

    WHEN SUM(od.quantity * od.unit_price) >= 3000
        THEN 'Average'

    ELSE 'Low'
END AS category_performance

FROM category_master c

JOIN product_master p
    ON c.category_id = p.category_id
```

JOIN order_details od
ON p.product_id = od.product_id

GROUP BY
c.category_id,
c.category_name;

SELECT *
FROM mv_category_sales_01
ORDER BY category_id;

	category_id [PK] character varying (5) ✓	category_name character varying (50) ✓	total_products_sold integer ✓	totalOrders integer ✓	unique_customers integer ✓	total_revenue numeric (12,2) ✓	avg_revenue_per_order numeric (12,2) ✓	category_performance character varying (20) ✓
1	C1	Electronics	5	4	3	12000.00	3000.00	Excellent
2	C2	Clothing	4	3	2	10000.00	3333.33	Excellent
3	C3	Books	3	1	1	1500.00	1500.00	Low